

Katechon: From Inference to Real-Time Content

Simon Judd

June 2, 2026

Abstract

Katechon develops a media platform whose primary unit is not an existing post but a generated channel: a live state object that turns external signals, user context, memory, and agent traces into commandable media and interface surfaces. The useful output is not a raw clip, image, answer, or render. It is a state transition that makes a channel fresher, more accurate, more legible, more actionable, more shareable, or easier to continue. This paper frames scaling around cost per useful state, $\kappa = \mathbb{E}[C]/\Pr(Y = 1)$, rather than raw token or media-generation cost. Two loops drive the model. The compute loop spends lower-cost inference on candidate search, critique, repair, personalization, and continuity. The user loop turns commands, corrections, engagement, shares, and forks into feedback that improves the attention policy that decides which states deserve future compute. If both loops lower κ , more inference compounds into better generated channels. The zero-latency horizon is the limit case: generated channels become live operational interfaces to modeled reality, not merely faster media.

Contents

1	Introduction	2
2	Channels	3
3	Attention Architecture	4
4	Compute and Inference Scaling	5
4.1	The Unit of Scale	6
4.2	Inference as Candidate Search	6
4.3	Users as Feedback Compute	7
4.4	The Channel Transition Operator	8
5	The Zero-Latency Limit	9
6	References	10

1 Introduction

The strategic question is simple: what happens to a platform like Katechon if the cost and latency of AI-generated video, image, voice, code, and critique continue to fall?

Key Claim

If inference can be converted into useful generated channel states, then falling inference cost does not merely make Katechon cheaper to run. It changes the product surface. The platform can attempt more variants, evaluate more outputs, personalize more surfaces, and turn more moments into channels.

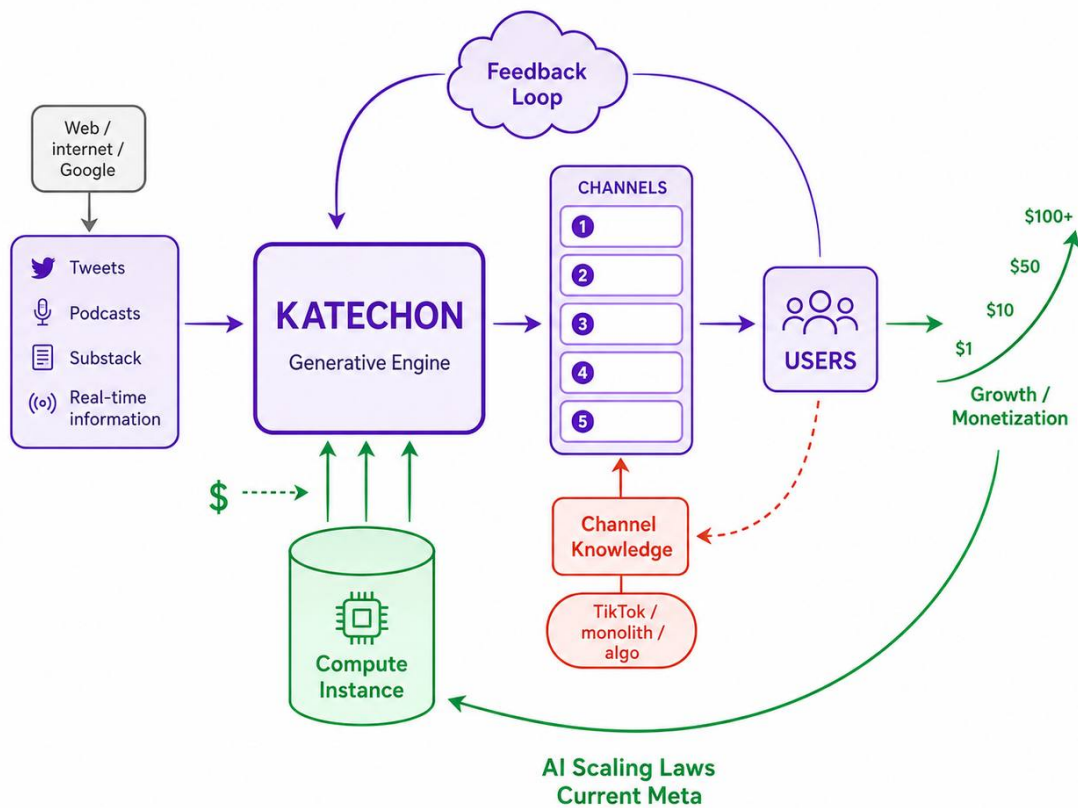


Figure 1: A first sketch of the Katechon scaling loop. External signals enter a generative engine, compute expands the search space, generated channels reach users, and feedback plus monetization funds the next round of useful state generation.

A generated channel is not only a feed or a playlist. It is a live, commandable, shareable state object that can contain generated media, software UI, market context, narration, user intent, memory, provenance, and an agent trace. It receives signals, chooses what deserves attention, renders a surface, and carries state forward so the next inference pass can continue rather than restart.

A useful state is a channel transition that improves at least one decision-relevant property: freshness, accuracy, legibility, emotional salience, actionability, shareability, or continuity. This defini-

tion makes the economic unit explicit. Katechon should not optimize for the cheapest token, the cheapest video, or the highest volume of generated artifacts in isolation. It should optimize for cost per useful state:

$$\kappa = \frac{\mathbb{E}[\text{full cost to produce and serve a state}]}{\Pr(\text{state is useful after critique and observation})}.$$

The denominator is the center of the thesis. More compute helps only when it raises the probability of a retained useful state faster than it raises the full cost of producing, evaluating, repairing, rendering, tracing, and serving that state inside the latency budget.

The platform has two compounding loops. The compute loop lowers κ by spending inference on candidate search, critique, repair, personalization, and continuity across states. The user loop lowers κ by turning commands, corrections, engagement, shares, forks, and follow-on actions into feedback for the attention policy: which channel state should exist next, how much inference it deserves, and how it should remain available for replay, sharing, forking, and action.

The zero-latency limit is the horizon, not the starting assumption. As generation, critique, repair, and rendering approach interactive speed, generated channels can become live interfaces to modeled reality. The output is no longer just more media. It is operational state: media that routes attention, supports decisions, preserves assumptions, exposes uncertainty, and remains ready for the next inference pass.

The buildable version is simple: define the channel state, measure whether each transition is useful, expose the knobs that change κ , and allocate inference where the next pass most improves the channel. Every aggressive scaling claim should be legible through those knobs.

2 Channels

A Katechon channel is not a passive feed and not a one-off generation. It is a live software object whose state is revised by inference. The channel receives fresh world data, user intent, memory, and context; chooses what deserves attention; then renders a mutable surface that can be watched, commanded, shared, forked, and continued. Each update produces a state capsule: a typed continuation object that builders can route, cache, replay, critique, repair, and use as the starting point for the next generation.

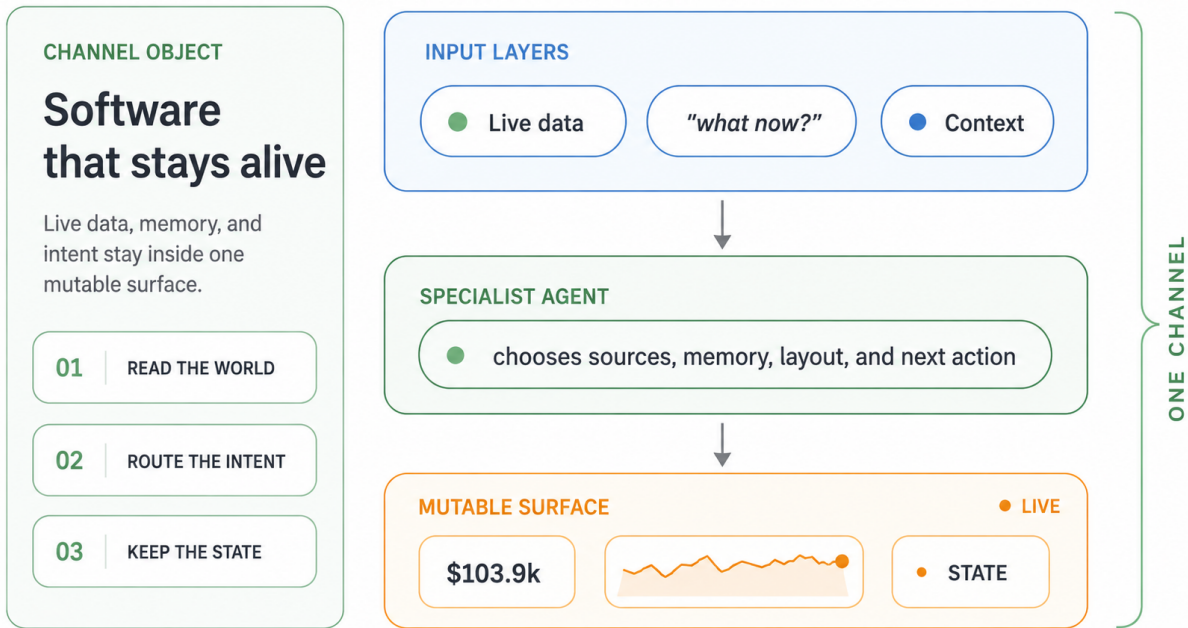


Figure 2: A generated channel is a live software object. Signals and intent update a persistent surface; each render carries enough state and trace to be watched, commanded, shared, forked, and continued.

For modeling purposes, write a channel as

$$\mathcal{H}_t = (x_t, m_t, g_t, u_t, \tau_t),$$

where x_t is live context, m_t is memory, g_t is the generated interface state, u_t is user intent, and τ_t is the agent trace. In a live market channel, for example, a price move, podcast clip, and user goal can become a chart, short explainer, and action surface with replayable provenance. Additional inference improves a channel only when it lowers the cost per useful state: the marginal render must make the next capsule more accurate, timely, legible, actionable, or easier to continue.

This definition gives the scaling model a product unit. The model should not count raw tokens or videos in isolation; it should count state transitions that survive critique and improve the channel.

3 Attention Architecture

The attention layer is the policy that decides which possible channel update deserves the next unit of inference. Its builder path is simple: signals become candidate states; candidate states are ranked and budgeted; selected candidates are generated, critiqued, and repaired; the output becomes a typed state capsule; and measured feedback updates the next turn.

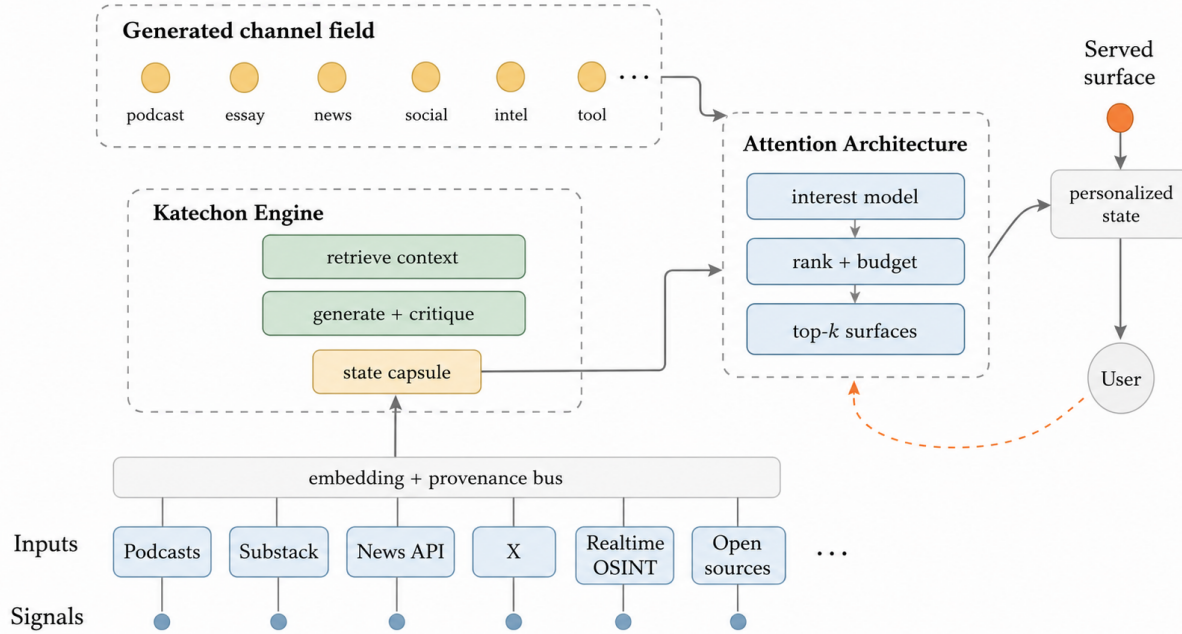


Figure 3: Katechon attention path. Signals enter with provenance, candidate states are generated and ranked, selected candidates are critiqued and repaired into typed state capsules, and served surfaces feed measured response back into the next attention step.

Let q_t be the channel's current intent query and let each candidate update i expose a key k_i , freshness score f_i , reliability score r_i , and expected utility $\hat{u}_i$. A minimal scoring rule is

$$a_i = \text{softmax}_i \left(\frac{q_t^\top k_i}{\sqrt{d}} + \lambda_f f_i + \lambda_r r_i \right), \quad i^* = \arg \max_i a_i \hat{u}_i.$$

The equation is deliberately small: semantic match, freshness, and reliability create an attention weight, and expected utility turns that weight into a choice. The selected candidate i^* becomes a generation budget, not merely a ranking result. The system allocates tokens, tool calls, media renders, retrieval, critique passes, and repairs until it can produce a state capsule that can be rendered and evaluated.

The key economic distinction is that attention quality compounds with generation quality. Better candidate selection reduces wasted renders; better critique and repair make each selected candidate more likely to become a useful state; better feedback updates the next attention query. This is the loop the scaling model should capture.

4 Compute and Inference Scaling

The core scaling claim is not that Katechon can make more media because media models become cheaper. The claim is stronger and more specific: if additional inference can be routed into search, critique, repair, personalization, and continuity across states, then each additional unit of compute

increases the probability that a channel produces a useful state for a specific user at a specific moment.

Key Claim

Katechon should be modeled as a test-time inference system over generated channel states. More compute expands the candidate set. More users improve the feedback distribution. Better feedback improves the attention policy. A better attention policy spends future compute on more valuable channel states.

This is the product-level form of the scaling thesis in Aschenbrenner (2024): count orders of magnitude of effective compute, algorithmic efficiency, and practical “unhobbling,” then ask what new behavior becomes possible. It is also the product-level form of Sutton (2019): build the system around general methods that can absorb more computation through search and learning, rather than around a fixed library of hand-authored channel formats.

4.1 The Unit of Scale

Raw tokens, frames, API calls, and videos are intermediate quantities. The unit that matters is a useful state transition.

Definition 4.1 (Useful state). A generated state transition s_{t+1} is useful for user u in channel j when it improves at least one decision-relevant property of the channel: freshness, accuracy, legibility, emotional salience, actionability, shareability, or continuity with prior state.

Let $Y(s, u, j, t) \in \{0, 1\}$ denote whether a candidate state is useful after critique and observation. Let C denote the full cost of producing and serving the state:

$$C = C_{\text{context}} + C_{\text{generate}} + C_{\text{critique}} + C_{\text{repair}} + C_{\text{render}} + C_{\text{trace}} + C_{\text{latency}}.$$

This cost is deliberately broader than the model invoice: it includes the context required to generate the state, the attempts needed to improve it, the trace needed to trust it, and the time penalty paid by the user. The economic primitive is cost per useful state,

$$\kappa_{j,u,t} = \frac{\mathbb{E}[C_{j,u,t}]}{\Pr(Y(s, u, j, t) = 1)}.$$

In words, κ asks how much total system cost Katechon pays for one state that is actually worth continuing, sharing, acting on, or remembering. Inference scaling helps only when it lowers κ : the useful-state probability must rise faster than total cost.

4.2 Inference as Candidate Search

For a given user and channel turn, Katechon can spend inference on K candidate states:

$$\mathcal{S}_K = \{s_1, \dots, s_K\}, \quad s_i \sim G_\theta(x_t, m_t, u_t, \mathcal{H}_t).$$

Here K is the number of attempts allocated to this turn. Each candidate has a latent usefulness probability ρ_i and an evaluator retention probability η_i . In a first-pass model that treats candidate

failures as independent, the probability of finding at least one retained useful state is

$$P_{\text{search}}(K) = 1 - \prod_{i=1}^K (1 - \rho_i \eta_i).$$

In the equal-candidate case, this simplifies to

$$P_{\text{search}}(K) = 1 - (1 - \rho\eta)^K.$$

The equation says that more candidates help only when the generator and evaluator are coupled: ρ raises the chance that a good state exists, and η raises the chance that the system keeps it. This is why the zero-latency media thesis is powerful. When generation, critique, and rerender latency fall, Katechon can move from one shot to best-of- K generation, then from best-of- K to structured search over formats, scripts, layouts, actions, and follow-up states.

The per-turn cost is approximately

$$C_K = C_{\text{context}} + K(C_{\text{generate}} + C_{\text{critique}}) + R_K C_{\text{repair}} + C_{\text{render}} + C_{\text{trace}} + C_{\text{latency}}(L_K),$$

where R_K is the expected number of repair passes and L_K is end-to-end latency. The immediate modeling target is therefore

$$\kappa_K = \frac{C_K}{P_{\text{search}}(K)}.$$

This turns the builder question into a budget question. The optimal K is not the largest affordable K ; it is the point where the next candidate, critique pass, or repair pass produces the largest marginal drop in κ_K under the current latency budget.

4.3 Users as Feedback Compute

Additional users do not only increase revenue or traffic. They increase the usable feedback signal available to the platform. Let $N_{j,t}$ be the number of active users in channel family j , $e_{u,j,t}$ the number of observable interactions from user u , and $w_{u,j,t}$ the reliability weight of that interaction stream. Define effective feedback density as

$$D_{j,t} = \sum_{u=1}^{N_{j,t}} w_{u,j,t} e_{u,j,t}.$$

This is not a raw engagement count. It is the amount of reliable signal the channel family can reuse when deciding what to generate next. The personalization and channel-fit term can be modeled as a saturating function:

$$F_j(D_{j,t}) = F_{j,\min} + (F_{j,\max} - F_{j,\min}) \left(1 - \exp(-\beta_j D_{j,t}^{\gamma_j}) \right), \quad 0 < \gamma_j \leq 1.$$

The curve says feedback compounds, but not forever. F_j improves as reliable signal accumulates, then saturates; γ_j exposes whether a channel family has dense, reusable signals or sparse, personal ones.

Combining search, personalization, and latency gives a first useful-state yield model:

$$\mathcal{U}_{j,u,t} = P_{\text{search}}(K_{j,u,t}) \cdot F_j(D_{j,t}) \cdot \exp\left(-\frac{\max(0, L_{j,u,t} - L^*)}{\tau_L}\right).$$

Here L^* is the interactive latency threshold and τ_L controls how quickly usefulness decays once a generated surface arrives too late. Operationally, the state is useful when search finds a good candidate, feedback makes it fit the channel, and the result arrives while the user can still use it.

4.4 The Channel Transition Operator

The feed-era algorithm selected from existing posts. Katechon’s attention algorithm chooses what generated state should exist next, how much inference it deserves, and how the resulting state should remain available for replay, sharing, forking, and action.

Let the policy be

$$\pi_\theta(a_t, b_t \mid u_t, m_t, x_t, \mathcal{H}_t),$$

where a_t is a candidate channel action and b_t is the inference budget assigned to that action. The objective is

$$\max_{\theta} \mathbb{E} \left[\sum_{t=0}^T \delta^t (U(s_{t+1}, u_t) - \lambda_C C_t - \lambda_L L_t - \lambda_R R_t) \right],$$

where U is user-state utility, C_t is cost, L_t is latency, and R_t is risk from hallucination, stale provenance, manipulative optimization, or low-quality generated media.

In words, the policy chooses both what to make and how hard to think before making it. A good policy spends more inference where the expected utility gain is high, the latency cost is tolerable, and the risk can be reduced through critique, repair, and provenance.

For the paper’s main display, the production loop should be written as a budgeted transition on channel state:

$$b_t = \pi_\theta(u_t, m_t, x_t, \mathcal{H}_t), \quad \mathcal{H}_{t+1} = \mathcal{T}_\theta(\mathcal{H}_t, x_t, u_t; b_t).$$

Here b_t is the inference budget assigned to this turn and $\mathcal{T}_\theta$ is the channel transition operator. Its implementation can be complex, but the displayed object is small:

$$\mathcal{T}_\theta = \text{Capsule} \circ \text{Select}_{\kappa, L, R} \circ \text{Repair} \circ \text{Critique} \circ \text{Generate}_{\theta, b_t}.$$

Read right to left: inference generates candidate states, critique scores them, repair converts near misses into usable states, selection keeps the best state under cost, latency, and risk constraints, and capsule construction commits the result as durable channel state.

Table 1: Operator view of one generated-channel turn.

Operator	Role	Economic effect
Generate	Sample candidate states from context, memory, user intent, and current channel state.	Raises search width K and candidate usefulness ρ .
Critique	Score candidates for freshness, fit, quality, actionability, provenance, and risk.	Raises evaluator retention η and reduces wasted renders.
Repair	Use critique outputs to revise or rerender while expected κ falls.	Converts near misses into retained useful states.
Select	Choose the candidate with the best useful-state yield under cost, latency, and risk.	Stops spending when marginal inference no longer lowers κ .
Capsule	Commit provenance, trace, replay, fork, action affordances, and updated memory.	Turns an output into durable state for the next transition.

Feedback is not a separate media-production step. It is the learning signal that updates the next policy call: user commands, shares, forks, corrections, engagement, and follow-on actions change future budgets, evaluator weights, and channel memory. Katechon does not win because it ranks preexisting media better. It wins if it learns, faster than competing systems, which state should be generated for which user, under which world conditions, with how much inference, and at what latency.

5 The Zero-Latency Limit

The zero-latency limit is the horizon at which signal, inference, rendering, and user action fit inside the latency budget of the moment. It does not mean omniscience or perfect simulation. It means fresh evidence can become a useful state before the opportunity to use that state expires.

In that limit, Katechon is not merely a faster video generator. It is a live channel generator: a system that constructs the most useful representation of an event, process, or goal from whatever evidence is available. A market move, a supply shock, a product launch, a sports sequence, a disaster zone, a public conversation, or an enterprise workflow can become a channel as soon as there is enough signal to form a useful state. A camera remains valuable, but it becomes one evidence stream among logs, text, user intent, sensors, prior state, and model outputs.

The radical but practical proposition is that media becomes an interface to modeled reality. The generated artifact does not substitute for the event. It is a state capsule: a visual, narrative, and interactive map that shows what matters, preserves assumptions, exposes uncertainty, and keeps enough provenance to be audited, replayed, shared, questioned, or forked. Its value is not fidelity alone; its value is whether it helps a user understand, decide, command, or continue.

For system builders, the zero-latency target is a contract for every generated state:

1. **Provenance:** what signals, sources, memories, and assumptions produced this state?
2. **Freshness:** is the state still inside the relevant latency budget?

3. **Usefulness:** does it improve the user’s ability to notice, understand, decide, act, share, or fork?
4. **Commandability:** can the user ask for a repair, expansion, comparison, counterfactual, or next action?
5. **Continuity:** can the next inference pass reuse the state, critique it, and update it without starting from scratch?

This is why zero latency belongs inside the cost-per-useful-state model. A state that arrives late has a lower useful-state probability. A state that arrives fast but loses provenance, trust, or action-ability is not useful either. The target is not minimum render time in isolation, but lower κ under a freshness, provenance, and action constraint.

As latency falls, a channel stops being content about an event and becomes the operating surface for the event. The relevant output is not “a clip.” It is an action-ready state that routes attention, supports decisions, triggers follow-up work, and remains available for the next inference pass. When measurable reality can be transformed into commandable channel state in real time, Katechon moves from scaling generated content to scaling live understanding.

6 References

- Aschenbrenner, Leopold (2024). *Situational Awareness: The Decade Ahead*. URL: <https://situational-awareness.ai/> (visited on 06/02/2026).
- McLuhan, Marshall (1964). *Understanding Media: The Extensions of Man*. McGraw-Hill.
- Snell, Charlie, Jaehoon Lee, Kelvin Xu, and Aviral Kumar (2024). *Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters*. DOI: [10.48550/arXiv.2408.03314](https://arxiv.org/abs/2408.03314). arXiv: 2408.03314 [cs.LG]. URL: <https://arxiv.org/abs/2408.03314>.
- Sutton, Richard S. (2019). *The Bitter Lesson*. URL: <http://www.incompleteideas.net/IncIdeas/BitterLesson.html> (visited on 06/02/2026).